

Constrained Decoding has limited performance uplift

Kartavya Vashishtha for CS 497

30 April, 2026

Key Points

- *Constrained decoding*
-
-

Key Points

- *Constrained decoding*
- a method showing promise
-

Key Points

- *Constrained decoding*
- a method showing promise
- seemingly falls short.

Key Points *(as a haiku)*

- *Constrained decoding*
- *a method showing promise*
- *seemingly falls short.*

LLMs are *auto-regressive*.

$\underbrace{\text{The}}_1 \quad \underbrace{\text{cat}}_2 \quad \underbrace{\text{jumped}}_3 \quad \underbrace{\text{over}}_4 \quad \underbrace{\text{the}}_5 \quad \bigcirc$

$$p(\text{fence} \mid \text{the cat jumped over the}) = 0.7$$

$$p(\text{cheese} \mid \text{the cat jumped over the}) = 0.1$$

Decoding: we sample from these probabilities to generate the next token.

Constrained Decoding: we mask out some undesirable tokens and then sample from the rescaled probabilities.

Prior Work in Constrained Decoding

	<completion>	Vanilla	Syntax	Types	Description
function is_int(text: string): boolean {	(1) ;	accept	reject	reject	Syntactically invalid
const num = Number(text);	(2) ber	accept	accept	reject	Undeclared identifier
return !isNaN(num) &&	(3) ()	accept	accept	reject	Disallowed operator
parseInt(num <completion>	(4) , 10)	accept	accept	reject	Invalid argument type
	(5) .toString()	accept	accept	accept	Well-formed option

Figure 1. Left is a partial TypeScript program derived from instance #113 of the MBPP benchmark [5], awaiting completion. Right are five completion options: (1)-(4) are invalid and (5) is well-formed. Our type-constrained decoding is the only approach capable of correctly rejecting invalid completions and accepting the valid one.

Intuitively: performance uplift

Table 3. pass@1 of unconstrained generation (Vanilla) and type constraining (Types). The benefit of our type-constraining approach transfers from reduced compilation errors to improved functional correctness.

	Model	Synthesis		Translation		Repair	
		Vanilla	Types	Vanilla	Types	Vanilla	Types
HumanEval	Gemma 2 2B	29.1	30.2 ↑3.8%	50.2	53.9 ↑7.5%	11.6	20.9 ↑79.4%
	Gemma 2 9B	56.6	58.3 ↑3.1%	73.7	78.3 ↑6.2%	24.0	34.9 ↑45.7%
	Gemma 2 27B	69.5	71.2 ↑2.5%	86.6	87.7 ↑1.3%	38.4	41.1 ↑7.1%
	DS Coder 33B	68.9	71.1 ↑3.2%	88.7	90.1 ↑1.6%	47.6	50.7 ↑6.5%
	CodeLlama 34B	41.0	43.4 ↑5.7%	58.6	63.5 ↑8.3%	17.5	27.4 ↑56.9%
	Qwen2.5 32B	79.6	81.8 ↑2.8%	92.1	93.9 ↑1.9%	65.4	71.2 ↑8.9%
MBPP	Gemma 2 2B	40.4	42.4 ↑5.2%	52.3	56.0 ↑7.0%	12.1	22.6 ↑86.7%
	Gemma 2 9B	65.4	67.4 ↑3.2%	71.4	75.8 ↑6.2%	24.2	31.9 ↑31.7%
	Gemma 2 27B	70.6	72.1 ↑2.2%	83.1	84.4 ↑1.6%	39.1	45.2 ↑15.5%
	DS Coder 33B	65.4	67.2 ↑2.8%	85.9	89.1 ↑3.6%	35.1	43.1 ↑23.0%
	CodeLlama 34B	42.2	45.6 ↑8.0%	55.7	63.3 ↑13.6%	15.7	26.6 ↑69.2%
	Qwen2.5 32B	76.3	76.6 ↑0.3%	89.6	90.4 ↑0.9%	48.0	54.0 ↑12.6%

Except newer models experience less uplift

Hypothesis: Current frontier models have nearly no benefit from constrained decoding. We validate this via BigCodeBench.

Model	Pass	Fail	Syntax	Type/API	Logical	Environ.	Timeout
Gemini Exp 1206	710 (62.3%)	430	0	30	388	10	2
DeepSeek-V3	708 (62.1%)	432	0	33	375	22	2
Gemini 2.0 Flash Exp	684 (60.0%)	456	0	41	397	16	2
GPT-4o (2024-11-20)	672 (58.9%)	468	0	41	403	22	2
Claude 3.5 Sonnet (Oct 2024)	658 (57.7%)	482	0	29	438	12	3
DeepSeek-R1-Distill-Llama-70B	571 (50.1%)	569	0	73	478	13	5
Aggregate	4003 (58.5%)	2837	0	247	2479	95	16

Table 1. Four-tier failure classification on BigCodeBench-Complete-Full (1,140 tasks per model). Syntax = Tier 1 (SyntaxError/IndentationError). Type/API = Tier 2 (TypeError/AttributeError). Logical = Tier 3 (AssertionError/ValueError/KeyError/etc.). Environ. = Tier 4 (FileNotFoundError/PermissionError/OSError/etc.). Models sorted by pass rate.